AMP Project Report Pages: 1-15 Submitted Dec. 5, 2025

NASCAR for CubeSats: A Comparison of Primer Vector Theory and Kinodynamic SST

Ian Faber IAFA3649@COLORADO.EDU
Department of Aerospace Engineering Sciences
University of Colorado Boulder

Abstract

The concept of CubeSat racing is revisited with an emphasis on optimal cubesat motion planning. A kinodynamic Sparse Stable RRT planning approach is explored and compared to the previous Primer Vector Theory approach, and it is found that kinodynamic SST planning, as implemented, does a generally worse job than primer vector theory. Trajectories and costs from both methods are compared from the same race course, and future improvements to the optimal planning problem are proposed.

1 Introduction

In a previous project, the idea of CubeSat racing was introduced from an optimal control perspective (Faber, 2025b). The idea was simple: Given a randomized course of waypoint rings and a randomized starting position, compute the optimal trajectory between waypoints that minimizes the time taken for a CubeSat to traverse the entire course while also minimizing the distance to the center of each ring. It was found that Primer Vector Theory (PVT) (Lawden, 2006) in context of the linearized Clohessy-Wiltshire Hill (CWH) equations could be leveraged to generate trajectories for each CubeSat and accomplish the goal.

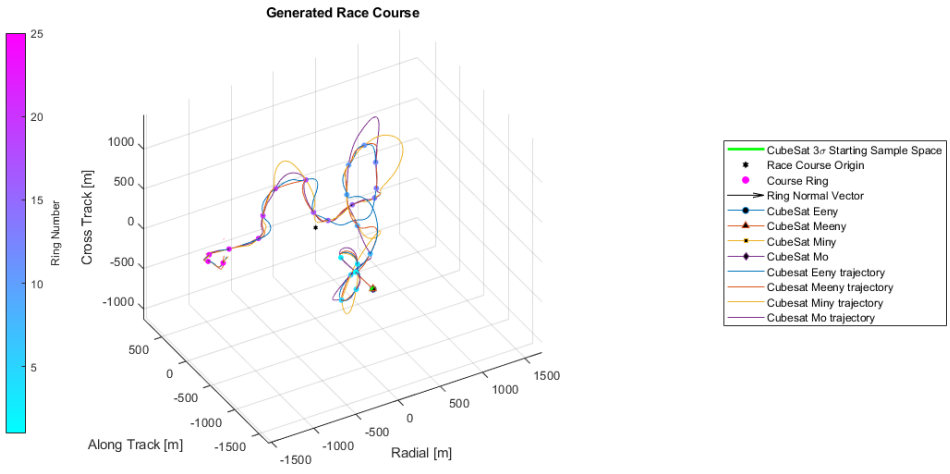

Figure 1: Example CubeSat Race

However, a major problem of the approach in Faber (2025b) is that the race course solving procedure took upwards of 30 minutes to an hour to solve each race course, and the resulting trajectories sometimes struggled to satisfy all the constraints – even leading to missing rings entirely. After learning about the Sparse Stable RRT algorithm (SST) in ASEN 5254, it was proposed as an alternate way to generate these optimal trajectories in potentially less time and with more constraint satisfaction. This project expands upon that idea by implementing SST to solve the same race course as the original project and compares it to the original PVT approach.

2 Problem Description

2.1 Problem Dynamics

For my implementation of this problem, I will be leveraging the CWH equations for the underlying dynamics model governing the CubeSats. The state to be controlled will be the typical relative motion state, arranged as follows:

$$\mathbf{x} = [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z} \ ]^\mathsf{T} \tag{1}$$

Where the origin (classically referred to as the "chief spacecraft") is the center of the race course. The CubeSats will have thrusters to control their x, y, and z accelerations like so:

$$\mathbf{u} = [u_x \ u_y \ u_z \ ]^\mathsf{T} \tag{2}$$

Where each CubeSat has a specified maximum thrust u_{max}. Thus, the dynamics of the system are as follows (Schaub and Junkins, 2018):

$$\mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ 2n\dot{y} + 3n^2 x + u_x \\ -2n\dot{x} + u_y \\ -n^2 z + u_z \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{a}_{\text{grav}} + \mathbf{u} \end{bmatrix} \tag{3}$$

Where n is the mean motion of the race course origin's orbit around Earth, x is the radial displacement, y is the along-track displacement, and z is the cross-track displacement of the CubeSat from the race course origin.

2.2 Race Course Generation

Every run of the CubeSat racing problem requires a randomly generated race course. Luckily, Faber (2025b) already accomplished this, and the method is repeated here for context.

A race course is made up of a sequence of rings that abide by the following physically realistic requirements:

1. Race course rings will be assumed to remain fixed in the Hill Frame (radial, along-track, cross-track), i.e. the CHW equations will not apply to the rings. This is a simplification to ensure that rings don't drift over the time period of the problem. For a relaxation of this assumption, see (Faber, 2025a).

2. Individual race course rings shall have semi-major and semi-minor axes uniformly distributed between $[1, 5]$ m.

3. Rings shall be spaced apart with uniformly distributed azimuth angles $\theta \in [-90, 90]^\circ$, elevation angles $\phi \in [-90, 90]^\circ$, and inter-ring distances $d \in [300, 350]$ m.

4. The first race course ring will be aligned along the +y axis.

5. Each race course shall consist of a uniformly distributed number of rings $l \in [15, 25]$ rings.

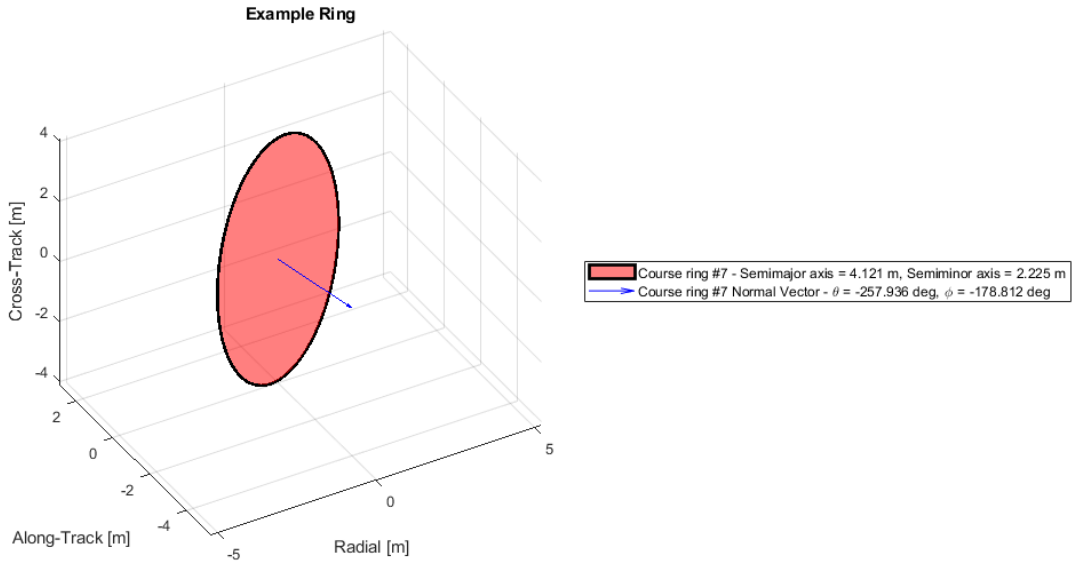

Figure 2: Example Race Course Ring

Once all rings are generated, the centroid of all the rings is calculated and set as the origin of the race course, which will act as the chief "spacecraft" for the CWH equations used in this project. Once all rings have been generated and the race course origin has been assigned, the result is a randomized race course as seen in Figure 3.

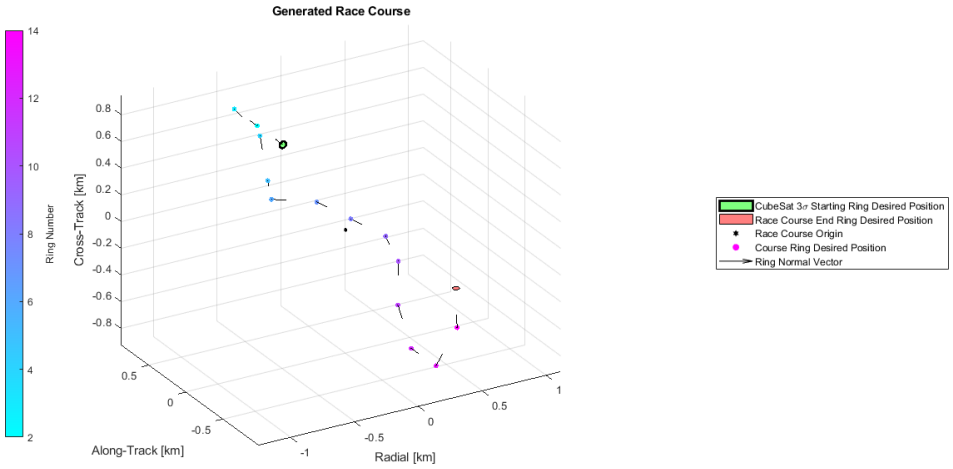

Figure 3: Example Randomly Generated Race Course

2.3 Cubesat Generation

Similarly, CubeSats must abide by the following set of rules:

1. CubeSat initial states shall be randomized according to a covariance of diag($[250, 0, 250]$) m^2 in x, y, and z respectively around a point exactly 350 m behind the first course ring. CubeSats shall be initially at rest.

2. CubeSat final states shall be at rest after passing through the last ring.

3. CubeSats will successfully pass through the center of a ring if their velocity vector at that ring is aligned with the normal vector of that ring (velocity has a positive projection onto the normal vector) and their final position is within the semi-minor axis of the ring.

2.4 Overall goal

The overall goal to be achieved by each CubeSat is to minimize the total race course traversal time while also minimizing the distance to the center of each race course ring.

3 Methodology

3.1 Tools

To solve this problem, I will be utilizing the Open Motion Planning Library (OMPL) (Şucan et al., 2012), in particular the OMPL implementation of SST. For plotting, I will be using existing MATLAB tools that were put together for Faber (2025b).

3.2 Problem Setup

To use OMPL's SST implementation, I needed to define the problem in a way that it could understand. In particular, I needed to define the workspace for the race course, the state space to plan over, the control space of valid controls to be applied, state validity checking for each candidate state, a structure for valid goal regions, the dynamics to be propagated, and finally the optimization objectives to determine the cost of each candidate node. Further, I had to set the tuning parameters for SST itself.

Workspace Definition

To define the workspace, I populated a YAML file with everything relevant to the current problem. This includes course dimensions, the mean motion of the race course origin, all relevant parameters for each ring, and all relevant parameters for each CubeSat.

All of these parameters were pulled from the original project in order to ensure a fair comparison between methods, namely, the exact same race course and CubeSat configurations. Once the YAML file was populated, I used a parsing script to create a "world" for the problem at hand.

State Space Definition

To define the state space, I leveraged OMPL's CompoundStateSpace type. I then populated it with two individual state spaces, namely, $\mathbb{R}^3$ for position and $\mathbb{R}^3$ for velocity. I set the bounds of the position state space to be the dimensions of the full race course, and I set the bounds of the velocity to be between -10 and 10 m/s, which is about the same range as reported by Faber (2025b). Further, I applied weights to each subspace due to the massive difference in magnitude between the positions (on the order of 1000 m) and velocities (on the order of 10 m/s). This allows for better sampling, as distances in the much smaller velocity space can be scaled to about equivalent to distances in the larger position space. In particular, the velocity space weight is calculated as follows:

$$w_v = \frac{\max\left((x_{max} - x_{min}), (y_{max} - y_{min}), (z_{max} - z_{min})\right)}{v_{max} - v_{min}}$$

This weight is generally larger than 1, as the race course can span upwards of 1000 m in any direction while admissible velocities only span 20 m/s in this case.

Control Space Definition

To define the control space, I leveraged OMPL's RealVectorControlSpace to implement an $\mathbb{R}^3$ control space. I then set the bounds on this control space to be whatever the maximum control effort was populated as in the workspace definition for each CubeSat.

State Validity Checking

The state validity checking is trivial for this problem, as there are no obstacles. Thus, the candidate state is valid as long as it lies within the bounds of the race course. This is an area that could be expanded in a future project, as primer vector theory has no way to account for obstacles like SST can.

Goal Region Definition

To define goal regions, I implemented a new class deriving from OMPL's GoalRegion class, which required me to overwrite the "distanceGoal" and "isSatisfied" methods. For this problem, "distanceGoal" is simply the distance between the CubeSat position and ring center. "isSatisfied" returns true if the distance from distanceGoal is less than the ring's semi-minor axis AND the projection of the CubeSat's velocity onto the ring's normal vector is positive, i.e. their dot product returns an angle less than 90 degrees.

Dynamics Definition

To define the dynamics to be propagated, I simply coded up a function that populates the vector $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$, given a current state vector $\mathbf{x}$ and proposed control vector $\mathbf{u}$. The function also requires the mean motion of the race course origin, which is easily passed in via a lambda function. The dynamics function is then interpreted by OMPL's ODESolver object, which runs an RK4 integration scheme.

Optimization Objective Definition

To define the optimization objective used by SST, I created a new class deriving from OMPL's OptimizationObjectivePtr class, which implements a MultiOptimizationObjective made up of a PathLengthOptimizationObjective (which for kinodynamic planners equates to control time duration) and a custom CenterObjective to optimize for distance to the center of each ring. The cost for CenterObjective is simply the distance between the CubeSat position vector and the center position of the ring. The individual objectives are weighted equally.

SST Parameter Definition

Finally, I defined the goal bias for SST to be decently high at 0.3 to force more sampling towards the target ring, selection radius of 15 m to more efficiently explore the 1000+ m dimensions of the workspace, and a pruning radius of 8 m to ensure that nodes are pruned effectively, but not so much that nodes which would pass through a ring would be discarded. This was a lot of trial and error, and I'm not convinced that my parameters are 100% correct for the problem. However, the resulting paths are passable, so I moved forward with them in the interest of time.

3.3 Solving the Problem

To solve the problem, I split up the overall race course into a sequence of smaller planning problems in between each ring. This mimics what was done for PVT, and it reduces the size of SST's workspace from the entire race course workspace to a smaller "segment workspace." I then ran SST on each segment for 60 seconds in sequence and concatenated all the final solutions together at the end. The reasoning for this is that if each individual segment is deemed to be optimal, then the overall concatenated path should also be optimal. There is probably a method for using SST with multiple goals to solve the problem all at once, but to keep the comparison fair with PVT I wanted to reproduce the approach used in Faber (2025b).

4 Results

After solving the same race course as Faber (2025b), the following trajectories were generated:

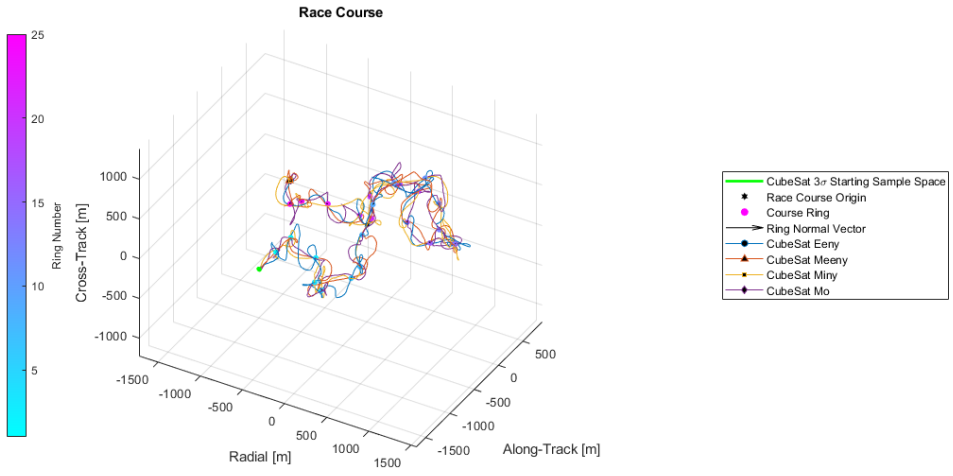

Figure 4: SST Race Course Solution

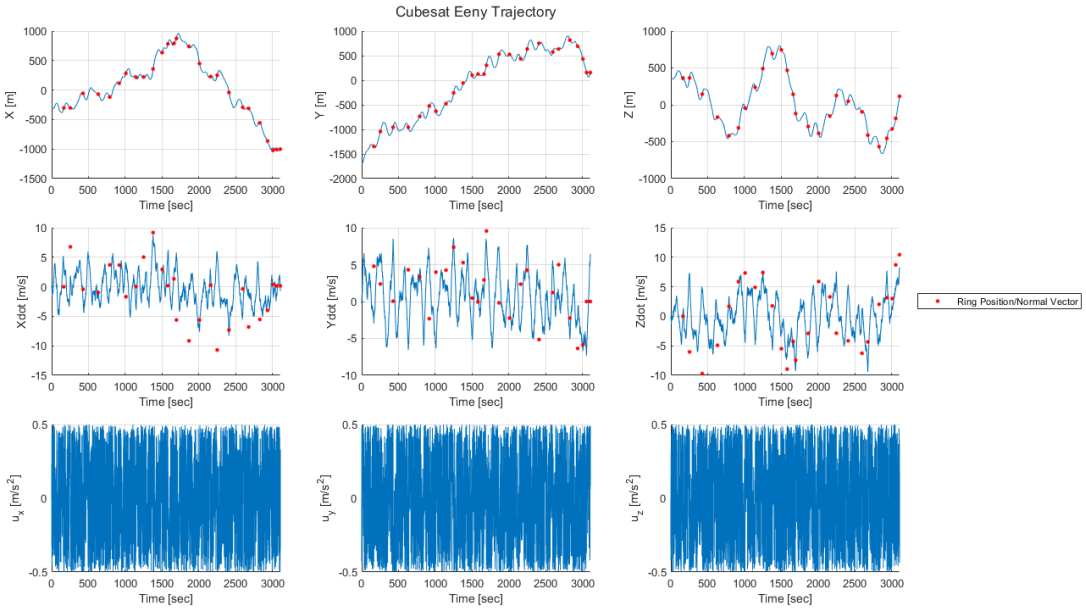

Figure 5: SST CubeSat Eeny Trajectory

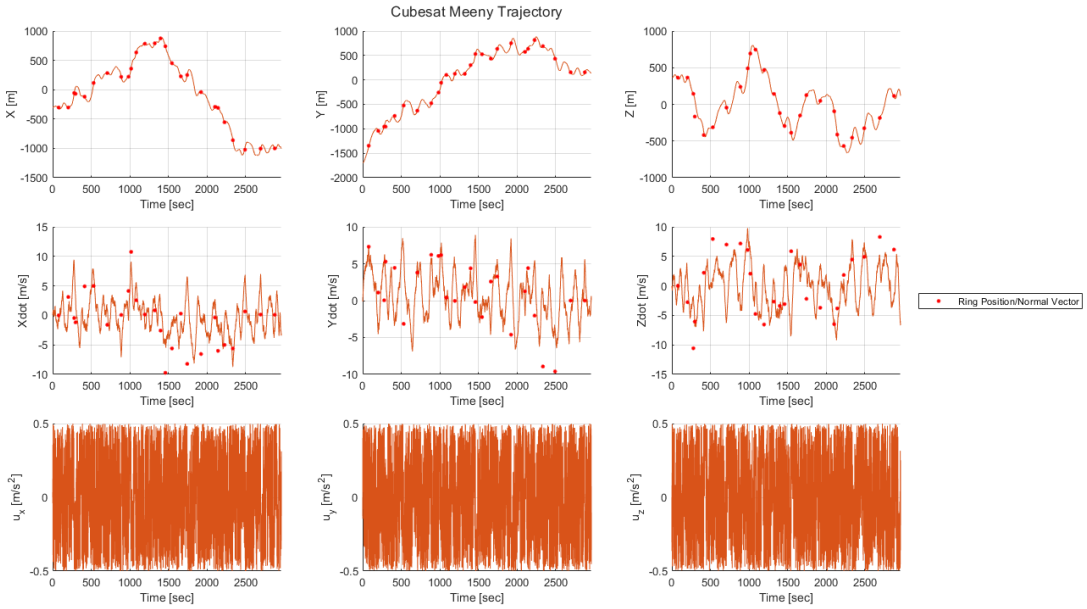

Figure 6: SST CubeSat Meeny Trajectory

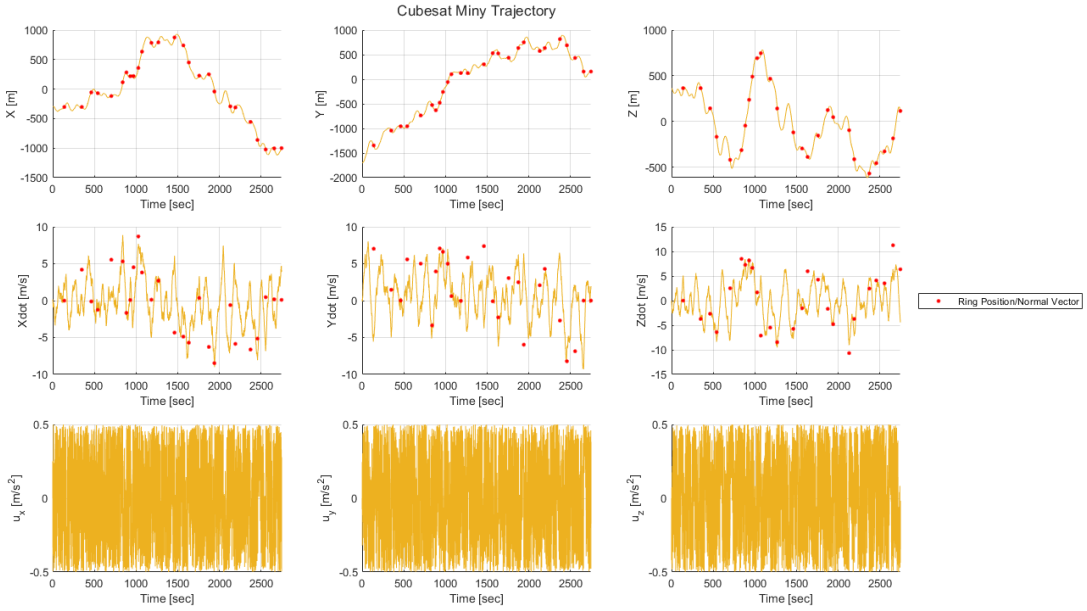

Figure 7: SST CubeSat Miny Trajectory

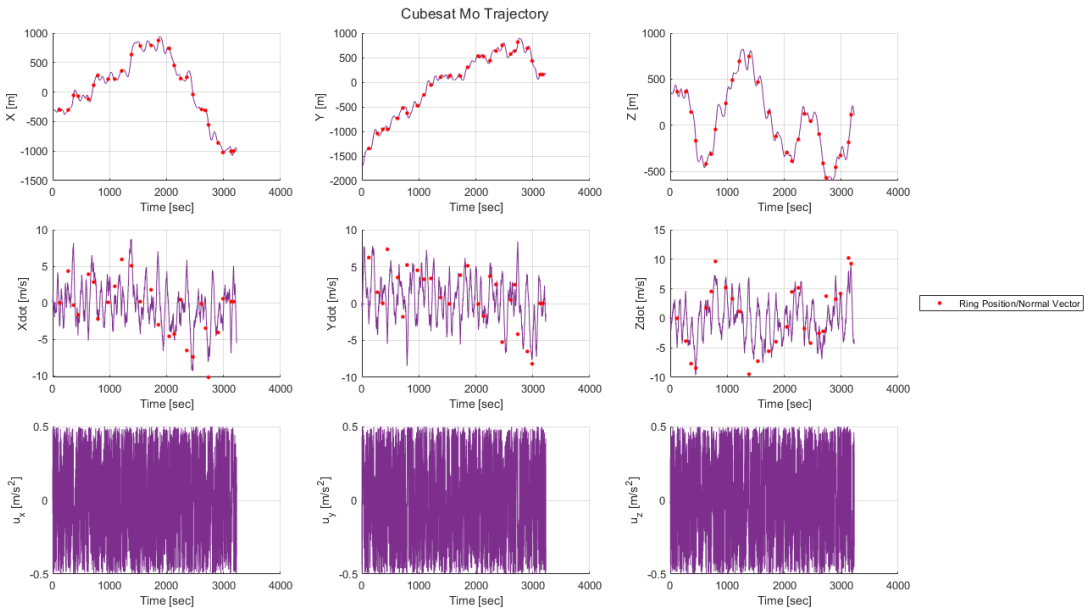

Figure 8: SST CubeSat Mo Trajectory

From Figure 4, we can see that while the generated paths are very erratic, they do generally follow the waypoints of the generated race course. This is further backed up by Figures 5 through 8, which show position states coinciding nicely with the ring centers, indicated by red dots along the trajectory.

However, just like PVT, SST struggles to satisfy the velocity pointing constraint. In fact, SST seems to struggle with it more than PVT – sometimes even flying backwards through a ring. The difference in path quality becomes even more apparent when we revisit the PVT results from Faber (2025b) in Figures 9 through 13, ignoring the middle adjoint plots:

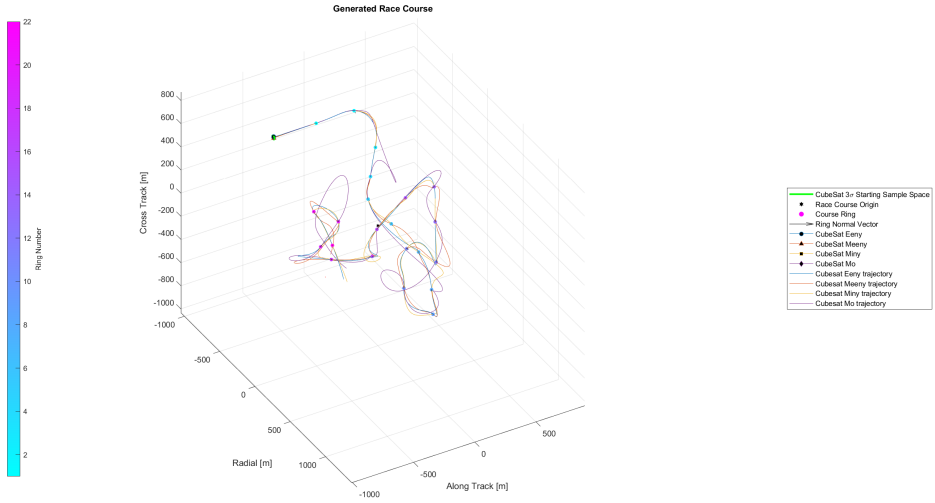

Figure 9: Primer Vector Theory Race Course Solution

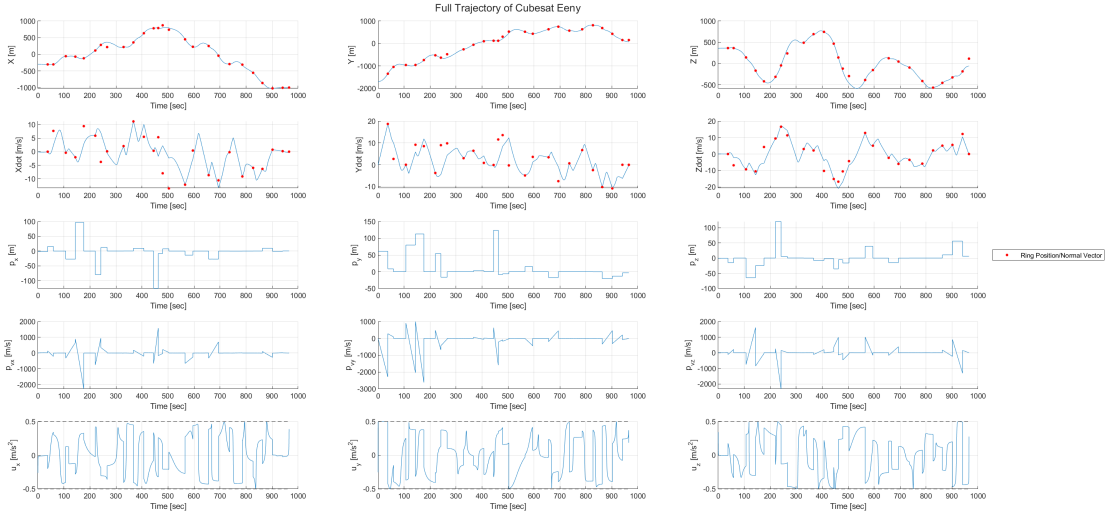

Figure 10: PVT CubeSat Eeny Trajectory

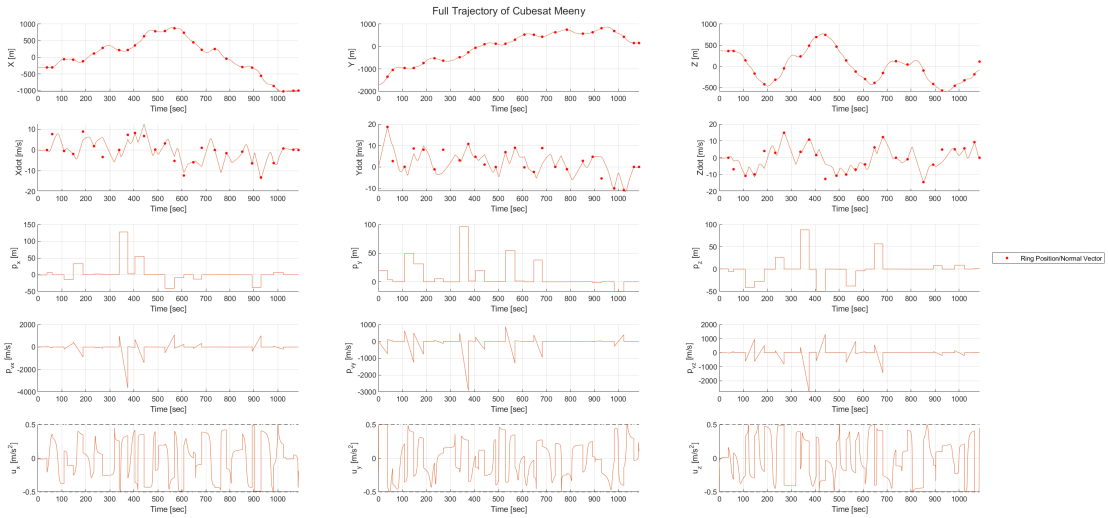

Figure 11: PVT CubeSat Meeny Trajectory

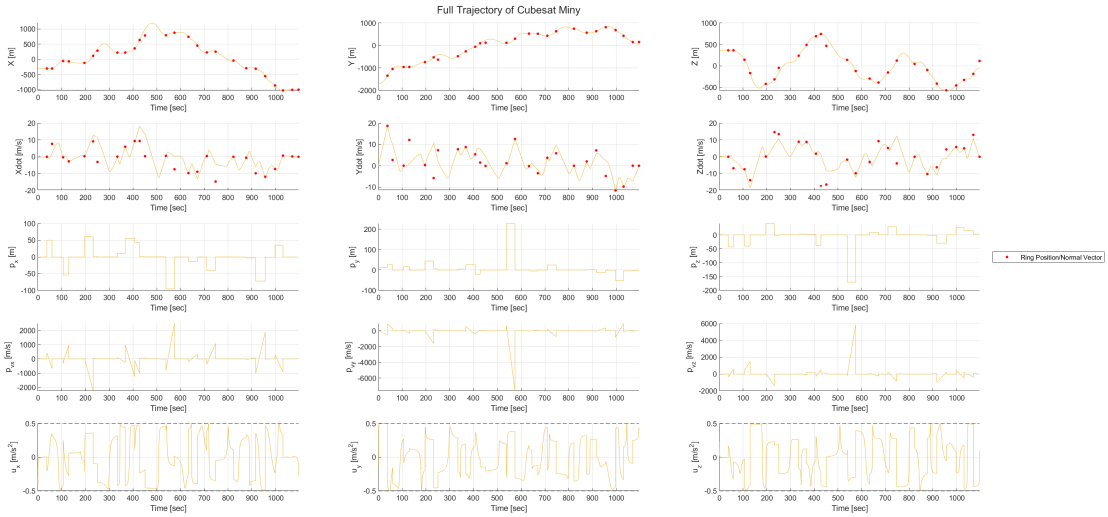

Figure 12: PVT CubeSat Miny Trajectory

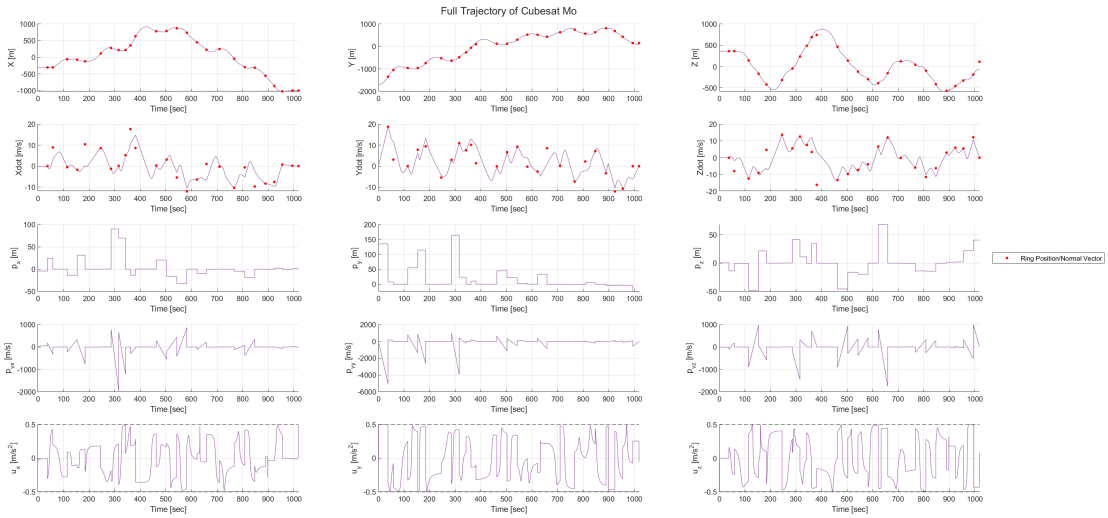

Figure 13: PVT CubeSat Mo Trajectory

The PVT results are clearly much smoother, and while a few points on the trajectory might deviate substantially from the waypoints, the CubeSats never fly backwards through a ring like in SST.

We can also compare the approaches through their final race course costs. The costs generated by SST and PVT are compared in Table 1, with the lowest cost in each category highlighted in yellow:

Table 1: Cost Comparison Between PVT and SST

CubeSat	**Overall Cost** (PVT, SST)	**Time** (PVT, SST) [sec]	**Accuracy** (PVT, SST) [m]
Eeny	1574.212, 3552.842	965.223, 3107.75	608.989, 445.092
Meeny	1133.655, 3699.124	1085.772, 2966.45	47.883, 732.674
Miny	1761.798, 3250.81	1096.043, 2746.85	665.755, 503.96
Mo	1161.586, 3677.403	1018.303, 3229.65	143.283, 447.753

Clearly, PVT always results in the lower cost. This is because PVT solves the course with much faster trajectories than SST, as the main finding from PVT is to always be thrusting at max acceleration. Meanwhile, SST only samples random controls with an upper bound of the max acceleration, which will de facto result in slower trajectories.

In addition, the solutions returned by SST are generally much less accurate than the solutions returned by PVT. While PVT occasionally results in a state that is very inaccurate, driving up its accuracy cost, on average the states generated by PVT are within a few meters of the goal ring. On the other hand, SST is consistently off by up to dozens of meters, which over 25 rings adds up to major inaccuracies.

However, the one positive of SST is that it does run faster than PVT. Since we cap SST's runtime at 60 seconds per segment, a 25 segment race course will always take 25

minutes to run. This is different from PVT, which can take up to an hour to run due to its gradient-descent based approach to finding trajectories.

5 Conclusion

In this project, Sparse Stable RRT and Primer Vector Theory were compared as solutions to the optimal CubeSat racing problem. It was found that, while SST does run faster than PVT, the paths it produces are much worse quality. The SST paths take much longer to traverse the course, and their ring accuracies are much worse on average. This could be due to subpar algorithm parameters, but the fundamental difference in control strategies between SST (randomly sample a control) and PVT (apply full throttle at an optimal direction) means that SST will always be at a disadvantage in terms of course traversal. For this reason, it would seem that PVT is the correct tool to use for this problem, despite the much longer computation time required.

Hope is not lost for SST though. One of the main drawbacks of PVT is how to initialize the adjoints used to determine the optimal control direction, and the method in Faber (2025b) uses a gradient descent approach to choose the right initial adjoints. As ASEN 5254 has illuminated, gradient descent can fall victim to local minima, which is what happens when PVT generates a very incorrect state. To alleviate this problem, SST could be used to initialize these adjoints instead of gradient descent, which would potentially speed up PVT as a whole and result in better paths. Thus, the true correct tool could be to combine the best parts of SST (sampling, speed) and PVT (path smoothness, course traversal time) to create the ultimate CubeSat race solver. Exciting times in the CubeSat racing world!

Acknowledgments

Special thanks to Dr. Scheeres for initially approving this deceptively fascinating project in the Optimal Trajectories course at CU Boulder, as well as to Dr. Lahijanian for approving me to continue to investigate it in the Algorithmic Motion Planning course at CU Boulder.

Appendix 1. Race Course Animations

To view animations of each solved race course considered in this report, visit the associated google drive link.

- SST Race Course Animation: `https://drive.google.com/file/d/1JK3hLeopTIhIa_nQXh8wXDfDOF92-0tL/view?usp=sharing`
- PVT Race Course Animation: `https://drive.google.com/file/d/12x3zqbFPNTMp4vRGmrBfVbysQisE view?usp=sharing`

Bibliography

I. Faber. Nascar for cubesats: Cubesat racing ring deployment and control, 2025a. URL https://drive.google.com/file/d/1qyOOK1Jz0BmMQiSBStBplIRcOO9Dwf8i/view?usp=sharing.

I. Faber. Nascar for cubesats: An application of primer vector theory, 2025b. URL https://github.com/EpicIan60142/NASCARforCubeSats.

D. F. Lawden. *Analytical Methods of Optimization*. Dover Publications Inc., first edition, 2006. ISBN 0-486-45034-1.

Hanspeter Schaub and John L. Junkins. *Analytical Mechanics of Space Systems*. AIAA, fourth edition, 2018. ISBN 9781624105210.

Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. The Open Motion Planning Library. *IEEE Robotics & Automation Magazine*, 19(4):72–82, December 2012. doi: 10.1109/MRA.2012.2205651. https://ompl.kavrakilab.org.